Product Requirements Document (PRD)

Ultra-Low Resource LLM Context Compression Engine

Hackathon: InnovaHacks — Generative AI Domain **Team Lead:** Yash Sethi **Version:** 1.0 **Timeline:** 24 hours

---

1. Problem Statement

Deploying large contextual histories (codebases, logs, chat history, documents) into LLM prompts creates heavy memory overhead, high latency, and steep API costs. A large portion of long context windows is repetitive syntax and filler boilerplate — wasting compute and slowing down real-time interactions.

2. Product Vision

Build an algorithmic + AI-assisted **pre-processing engine** that sits between the user's raw context and the target LLM. It strips semantic redundancy, compresses the prompt by 70%+, and preserves 95%+ of downstream answer accuracy — proven live via a side-by-side accuracy comparison.

3. Target Users

- Developers feeding large codebases into LLM prompts (Cursor/Copilot-style tools)
- Companies running customer support bots on long chat histories
- Anyone paying per-token for LLM API calls with long context needs

4. Goals & Success Metrics

Goal	Metric	Target
Reduce prompt size	Compression ratio	≥ 70% reduction
	Downstream answer similarity (compressed vs full	

Preserve accuracy	context)	≥ 95%
Reduce cost	Estimated $ saved per prompt (token-based)	Shown live in demo
Reduce latency	Response time improvement	Shown live in demo
Demo credibility	Live before/after accuracy proof	Working end-to-end

5. Core Features (MVP Scope for 24 Hours)

1. **Input intake** — paste or upload text (code file, log file, or chat transcript)
2. **Two-stage compression pipeline**
    - Stage 1: Structural/redundancy stripping (no LLM call — cheap & fast)
    - Stage 2: Semantic compression (LLM-assisted, preserves high-signal content)
3. **Compression dashboard** — shows original vs compressed token count, % reduction, estimated cost saved
4. **Accuracy validation loop** — ask a test question against (a) full context and (b) compressed context using Claude API, show both answers side-by-side for the judges to compare
5. **Before/after diff view** — visually shows what was removed/compressed

6. Out of Scope (for 24-hr MVP)

- Multi-modal input (audio, images, schematics)
- User authentication / accounts
- Persistent storage across sessions (beyond basic history for the demo)
- Support for files >5MB
- Fine-tuned custom compression model (we use prompting, not training)

7. User Flow

1. User lands on the app → pastes/uploads context (e.g., a 5,000-line log file)
2. User optionally types a test question relevant to that context
3. User clicks “Compress”
4. App runs Stage 1 (structural strip) → shows intermediate token count
5. App runs Stage 2 (semantic compression via Claude) → shows final token count + compression %
6. App runs the test question against both full and compressed context → shows both answers + a similarity score
7. Dashboard displays: compression ratio, cost saved, latency improvement, accuracy retention

8. Assumptions

- Claude API is used both for compression (Stage 2) and for the accuracy-validation Q&A step
- Demo inputs will be text-based (code, logs, or long text) — no binary/multi-modal files
- Judges will care most about: a working live demo, clear numbers, and the accuracy-proof step

9. Team Structure

Team	Responsibility
Frontend	Next.js UI — input, dashboard, diff view, results display
Backend	API routes, compression pipeline orchestration, Claude API integration
AI/ML	Compression algorithm design, prompt engineering, chunk scoring logic
Database	Session/history storage, chunk metadata, evaluation logs

10. Risks

Risk	Mitigation
LLM compression is too slow for	Use Stage 1 structural strip as fallback; parallelize

demo	chunk compression calls
Compression removes something important, breaks accuracy	Chunk-level scoring keeps entities/numbers/code intact; test on multiple samples before demo
Running out of time	MVP fallback = dedup + Claude summarization per chunk only, skip fancy scoring